In [1]:
```
import fastf1
import pandas as pd
import numpy as np

# use the data stored locally
fastf1.Cache.enable_cache('../data/raw')

# load the race - bahrain
session = fastf1.get_session(2025, 4, 'R')
session.load()

laps = session.laps

# take safety car laps
is_sc_vsc = laps['TrackStatus'].str.contains('4|6', na=False, regex=True)
# Assign fuel burn: 0.8kg for SC/VSC laps, 1.5kg for normal laps
laps['Fuel_Burn_Per_Lap'] = np.where(is_sc_vsc, 0.8, 1.5)
# Calculate cumulative fuel burn per driver
laps['Cumulative_Fuel_Burn'] = laps.groupby('Driver')['Fuel_Burn_Per_Lap'].c
# Set starting fuel to 110kg and subtract the cumulative burn
laps['Estimated_Fuel_Mass'] = 110.0 - laps['Cumulative_Fuel_Burn']

clean_laps = laps.copy()

# drop incomplete/inaccurate laps - safety car laps, pit laps
clean_laps = clean_laps[clean_laps['IsAccurate'] == True]
clean_laps = clean_laps[clean_laps['PitInTime'].isnull() & clean_laps['PitOu
clean_laps = clean_laps[~clean_laps['TrackStatus'].str.contains('4|6', na=Fa

# create Stint_ID - Cross-Validation Grouping Key
# a 'STINT' is a continuous run on one set of tires
# combine the Driver's 3-letter code with their integer Stint number
clean_laps['Stint_ID'] = clean_laps['Driver'] + "_Stint_" + clean_laps['Stir

# define the target variable
clean_laps['Target_Compound'] = clean_laps['Compound']



# convert LapTime from ns to s
clean_laps['LapTime_Sec'] = clean_laps['LapTime'].dt.total_seconds()

top3_avg_per_lap = (
    clean_laps.groupby('LapNumber')['LapTime_Sec']
    .nsmallest(3)                   # Get top 3 fastest times
    .groupby('LapNumber')           # Regroup by LapNumber
    .mean()                         # Average those 3 times
    .reset_index()
)
top3_avg_per_lap.rename(columns={'LapTime_Sec': 'Top3_Avg_Sec'}, inplace=Tru

# 2. Sort by LapNumber to ensure chronological order before smoothing
top3_avg_per_lap = top3_avg_per_lap.sort_values('LapNumber')
```

```
# 3. Apply Exponential Moving Average (EMA) to smooth out spikes
# span=5 means the EMA gives the most weight to the last 5 laps (~7-10 mins
top3_avg_per_lap['Field_Best_Lap_Sec'] = top3_avg_per_lap['Top3_Avg_Sec'].ew

# 4. Keep only the necessary columns for the merge
smoothed_baseline = top3_avg_per_lap[['LapNumber', 'Field_Best_Lap_Sec']]

# 5. Merge the smoothed baseline back into clean_laps dataframe
clean_laps = clean_laps.merge(smoothed_baseline, on='LapNumber', how='left')

# 6. Calc DELTA: how much slower was this driver compared to the smoothed tr
clean_laps['Delta_to_Field_Best'] = clean_laps['LapTime_Sec'] - clean_laps['

print(f"Original laps: {len(laps)} | Clean laps remaining: {len(clean_laps)}
print(clean_laps.head())
```

```
core           INFO     Loading data for Bahrain Grand Prix - Race [v3.8.1]
req            INFO     Using cached data for session_info
req            INFO     Using cached data for driver_info
req            INFO     Using cached data for session_status_data
req            INFO     Using cached data for lap_count
req            INFO     Using cached data for track_status_data
req            INFO     Using cached data for _extended_timing_data
req            INFO     Using cached data for timing_app_data
core           INFO     Processing timing data...
req            INFO     Using cached data for car_data
req            INFO     Using cached data for position_data
req            INFO     Using cached data for weather_data
req            INFO     Using cached data for race_control_messages
core           INFO     Finished loading data for 20 drivers: ['81', '63',
'4', '16', '44', '1', '10', '31', '22', '87', '12', '23', '6', '7', '14', '3
0', '18', '5', '55', '27']
```

```
Original laps: 1128 | Clean laps remaining: 952
                    Time Driver DriverNumber                LapTime  \
0 0 days 00:58:51.319000    PIA           81 0 days 00:01:37.492000
1 0 days 01:00:29.402000    PIA           81 0 days 00:01:38.083000
2 0 days 01:02:07.535000    PIA           81 0 days 00:01:38.133000
3 0 days 01:03:45.578000    PIA           81 0 days 00:01:38.043000
4 0 days 01:05:23.941000    PIA           81 0 days 00:01:38.363000

   LapNumber  Stint PitOutTime PitInTime            Sector1Time  \
0        2.0    1.0        NaT       NaT 0 days 00:00:31.139000
1        3.0    1.0        NaT       NaT 0 days 00:00:31.306000
2        4.0    1.0        NaT       NaT 0 days 00:00:31.326000
3        5.0    1.0        NaT       NaT 0 days 00:00:31.305000
4        6.0    1.0        NaT       NaT 0 days 00:00:31.336000

             Sector2Time  ... FastF1Generated IsAccurate Fuel_Burn_Per_Lap
\
0 0 days 00:00:42.343000  ...           False       True               1.5
1 0 days 00:00:42.727000  ...           False       True               1.5
2 0 days 00:00:42.796000  ...           False       True               1.5
3 0 days 00:00:42.690000  ...           False       True               1.5
4 0 days 00:00:42.880000  ...           False       True               1.5

  Cumulative_Fuel_Burn  Estimated_Fuel_Mass     Stint_ID  Target_Compound  \
0                  3.0                107.0  PIA_Stint_1             SOFT
1                  4.5                105.5  PIA_Stint_1             SOFT
2                  6.0                104.0  PIA_Stint_1             SOFT
3                  7.5                102.5  PIA_Stint_1             SOFT
4                  9.0                101.0  PIA_Stint_1             SOFT

   LapTime_Sec  Field_Best_Lap_Sec Delta_to_Field_Best
0       97.492           97.739667           -0.247667
1       98.083           97.910667            0.172333
2       98.133           98.011667            0.121333
3       98.043           98.117778           -0.074778
4       98.363           98.234074            0.128926

[5 rows x 39 columns]
```

In [2]:

```
################################################################## TELEMTF

telemetry_features = []

# loop through clean laps to get the high-frequency telemetry
for index, lap in clean_laps.iterrows():

    try:
        tel = lap.get_telemetry()
    except Exception as e:
        continue

    # calc basic aggregations
    avg_speed = tel['Speed'].mean()
    std_dev_speed = tel['Speed'].std()
    min_speed = tel['Speed'].min() # abs slower point on track = thightest h
    corner_speed_10th = tel['Speed'].quantile(0.10) # proxy for low-speed co
```

```
    avg_rpm = tel['RPM'].mean()
    avg_throttle = tel['Throttle'].mean()

    percent_full_throttle = (tel['Throttle'] >= 99).mean()

    # BRAKE EVENTS = how many times the brake pedal goes from 0 to >0
    brake_transitions = (tel['Brake'] > 0).astype(int).diff()
    brake_events = (brake_transitions == 1).sum()

    # Dirty Air / Traffic Proxy
    # 1. Convert Speed from km/h to m/s (to calculate time gap safely)
    # We replace 0 with 0.001 to avoid a "Division by Zero" error just in ca
    speed_ms = (tel['Speed'] / 3.6).replace(0, 0.001)

    # 2. Calculate dynamic time gap to the car ahead (Distance / Speed = Tim
    # This creates a moment-by-moment time gap for the entire lap
    tel['Time_Gap_Ahead'] = tel['DistanceToDriverAhead'] / speed_ms

    # 3. Create a mask for when the car is within 2.5 seconds (Dirty Air Zon
    # We use > 0 to filter out NaNs or errors when no car is ahead
    in_dirty_air = (tel['Time_Gap_Ahead'] > 0) & (tel['Time_Gap_Ahead'] <= 2

    # 4. Proxy for "Sliding / Cornering in Dirty Air"
    # Condition: In Dirty Air AND Speed < 250 km/h (cornering) AND Throttle
    dirty_air_cornering = in_dirty_air & (tel['Speed'] < 250) & (tel['Throtti

    # 5. Extract the features: What percentage of the lap was spent in these
    percent_in_dirty_air = in_dirty_air.mean()
    percent_dirty_air_cornering = dirty_air_cornering.mean()

    # add extracted features to the original ones
    telemetry_features.append({
        'Index': index,
        'Avg_Speed': avg_speed,
        'Std_Dev_Speed': std_dev_speed,
        'Min_Speed': min_speed,
        'Corner_Speed_10th': corner_speed_10th,
        'Avg_RPM': avg_rpm,
        'Avg_Throttle': avg_throttle,
        'Percent_Full_Throttle': percent_full_throttle,
        'Brake_Events': brake_events,
        'Percent_Dirty_Air': percent_in_dirty_air,
        'Dirty_Air_Cornering': percent_dirty_air_cornering
    })

# convert to dataframe object
tel_df = pd.DataFrame(telemetry_features)
tel_df.set_index('Index', inplace=True)

# merge telemetry features back into clean_laps DataFrame
clean_laps = clean_laps.join(tel_df)

# drop rows where telemetry was missing
clean_laps.dropna(subset=['Avg_Speed'], inplace=True)

# CALC this before cleaning laps to take into account safety laps!!!!!
```

```
# estimate fuel mass feature assuming linear burn rate
# clean_laps['Estimated_Fuel_Mass'] = 110.0 - (clean_laps['LapNumber'] * 1.5

# Degradation Derivative with 3-lap rolling average of the pace delta
clean_laps['Degradation_Slope'] = (
    clean_laps.groupby('Stint_ID')['Delta_to_Field_Best']
    .transform(lambda x: x.rolling(window=3, min_periods=1).mean())
)

# Weather data needs to be sorted by time for an 'asof' merge = merge by nea
weather_data = session.weather_data
weather_data = weather_data.sort_values(by='Time')
clean_laps = clean_laps.sort_values(by='LapStartTime')

weather_subset = weather_data[['Time', 'TrackTemp', 'AirTemp']].rename(colum

clean_laps = pd.merge_asof(
    clean_laps,
    weather_subset,
    # weather_data[['Time', 'TrackTemp', 'AirTemp']],
    left_on='LapStartTime',
    right_on='Weather_Time',
    direction='backward'
)

clean_laps.drop(columns=['Weather_Time'], inplace=True)

print(f"database final shape: {clean_laps.shape}")
print(clean_laps.columns)
```

```
database final shape: (952, 52)
Index(['Time', 'Driver', 'DriverNumber', 'LapTime', 'LapNumber', 'Stint',
       'PitOutTime', 'PitInTime', 'Sector1Time', 'Sector2Time', 'Sector3Tim
e',
       'Sector1SessionTime', 'Sector2SessionTime', 'Sector3SessionTime',
       'SpeedI1', 'SpeedI2', 'SpeedFL', 'SpeedST', 'IsPersonalBest',
       'Compound', 'TyreLife', 'FreshTyre', 'Team', 'LapStartTime',
       'LapStartDate', 'TrackStatus', 'Position', 'Deleted', 'DeletedReaso
n',
       'FastF1Generated', 'IsAccurate', 'Fuel_Burn_Per_Lap',
       'Cumulative_Fuel_Burn', 'Estimated_Fuel_Mass', 'Stint_ID',
       'Target_Compound', 'LapTime_Sec', 'Field_Best_Lap_Sec',
       'Delta_to_Field_Best', 'Avg_Speed', 'Std_Dev_Speed', 'Min_Speed',
       'Corner_Speed_10th', 'Avg_RPM', 'Avg_Throttle', 'Percent_Full_Throttl
e',
       'Brake_Events', 'Percent_Dirty_Air', 'Dirty_Air_Cornering',
       'Degradation_Slope', 'TrackTemp', 'AirTemp'],
      dtype='object')
```

In [4]:

```
# choose features to keep
# features = [
#     'Delta_to_Field_Best', 'Avg_Speed', 'Std_Dev_Speed', 'Min_Speed', 'Cor
#     'Avg_Throttle', 'Percent_Full_Throttle', 'Brake_Events', 'DRS_Active',
#     'Estimated_Fuel_Mass', 'Degradation_Slope', 'TrackTemp', 'TyreLife', '
# ]
features = [
```

```
    'Delta_to_Field_Best', 'Avg_Speed', 'Std_Dev_Speed', 'Min_Speed', 'Corne
    'Avg_Throttle', 'Percent_Full_Throttle', 'Brake_Events', 'Percent_Dirty_
    'Estimated_Fuel_Mass', 'Degradation_Slope', 'TrackTemp', 'TyreLife'
]

metadata = ['Target_Compound', 'Stint_ID']
clean_laps_fresh = clean_laps[metadata + features].copy()

# ONLY WHEN MORE RACES ARE AVAILABLE
# 3. Convert the boolean 'FreshTyre' column to integers (1 = Fresh, 0 = Scru
# XGBoost requires numeric features, so we map True/False to 1/0
# clean_laps_fresh['FreshTyre'] = clean_laps_fresh['FreshTyre'].astype(int)

# drop NaN values
model_df = clean_laps_fresh.copy()
model_df.dropna(inplace=True)

print(f"final modeling database shape: {model_df.shape}")
print(model_df.columns)
# pd.crosstab(model_df['Target_Compound'], model_df['FreshTyre'])
```

```
final modeling database shape: (952, 17)
Index(['Target_Compound', 'Stint_ID', 'Delta_to_Field_Best', 'Avg_Speed',
       'Std_Dev_Speed', 'Min_Speed', 'Corner_Speed_10th', 'Avg_RPM',
       'Avg_Throttle', 'Percent_Full_Throttle', 'Brake_Events',
       'Percent_Dirty_Air', 'Dirty_Air_Cornering', 'Estimated_Fuel_Mass',
       'Degradation_Slope', 'TrackTemp', 'TyreLife'],
      dtype='object')
```

In [6]:

```
############################################ DATABASE MODELING VERIFICATION
import os

# check data types and missing values
print(model_df.info())
print("-" * 30)
print("Missing values per column:\n", model_df.isnull().sum())
print("-" * 30)

os.makedirs('../data/cleaned', exist_ok=True)
model_df.to_csv('../data/cleaned/model_df.csv', index=False)
```

```
<class 'fastf1.core.Laps'>
RangeIndex: 952 entries, 0 to 951
Data columns (total 17 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Target_Compound        952 non-null    object
 1   Stint_ID               952 non-null    object
 2   Delta_to_Field_Best    952 non-null    float64
 3   Avg_Speed              952 non-null    float64
 4   Std_Dev_Speed          952 non-null    float64
 5   Min_Speed              952 non-null    float64
 6   Corner_Speed_10th      952 non-null    float64
 7   Avg_RPM                952 non-null    float64
 8   Avg_Throttle           952 non-null    float64
 9   Percent_Full_Throttle  952 non-null    float64
 10  Brake_Events           952 non-null    int64
 11  Percent_Dirty_Air      952 non-null    float64
 12  Dirty_Air_Cornering    952 non-null    float64
 13  Estimated_Fuel_Mass    952 non-null    float64
 14  Degradation_Slope      952 non-null    float64
 15  TrackTemp              952 non-null    float64
 16  TyreLife               952 non-null    float64
dtypes: float64(14), int64(1), object(2)
memory usage: 126.6+ KB
None
------------------------------
Missing values per column:
 Target_Compound          0
Stint_ID                 0
Delta_to_Field_Best      0
Avg_Speed                0
Std_Dev_Speed            0
Min_Speed                0
Corner_Speed_10th        0
Avg_RPM                  0
Avg_Throttle             0
Percent_Full_Throttle    0
Brake_Events             0
Percent_Dirty_Air        0
Dirty_Air_Cornering      0
Estimated_Fuel_Mass      0
Degradation_Slope        0
TrackTemp                0
TyreLife                 0
dtype: int64
------------------------------
```

In [7]:

```
############################################################## CROSS-VALIDA

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.metrics import f1_score
from sklearn.utils.class_weight import compute_sample_weight
import xgboost as xgb
import catboost as cb
import lightgbm as gbm
```

```
import optuna
import numpy as np

# CUSTOM WEIGHTS FOR THE COMPOUNDS
custom_weights = {0: 1.5, 1: 3.5, 2: 1.0}

# encode target variables: hard, medium, soft -> 0, 1, 2
le = LabelEncoder()
model_df['Target_Encoded'] = le.fit_transform(model_df['Target_Compound'])
label_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(f"Class mapping: {label_mapping}")

X = model_df[features]
y = model_df['Target_Encoded']
groups = model_df['Stint_ID']


# define the Optuna obj function for XGBOOST
def obj_xgboost(trial):

    param = {
        'objective': 'multi:softmax',
        'num_class': 3,
        'tree_method': 'hist',
        'random_state': 42,
        'max_depth': trial.suggest_int('max_depth', 3, 7),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 15),
        'gamma': trial.suggest_float('gamma', 1e-8, 1.0, log=True),
        'lambda': trial.suggest_float('lambda', 1e-3, 10.0, log=True),
        'alpha': trial.suggest_float('alpha', 1e-3, 10.0, log=True)
    }

    # use StratifiedGroupKFold to prevent stint leakage
    # all laps from a single STINT_ID stay together in either training or va
    sgkf = StratifiedGroupKFold(n_splits=5)
    f1_scores = []

    for train_idx, val_idx in sgkf.split(X, y, groups):
        X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

        # sample_weights = compute_sample_weight(class_weight=custom_weights
        sample_weights = compute_sample_weight(class_weight='balanced', y=y_

        # initialize and train XGBoost classifier model
        # use dynamic sample weights for class imbalances
        model = xgb.XGBClassifier(**param)
        model.fit(
            X_train, y_train,
            sample_weight=sample_weights,
            eval_set=[(X_val, y_val)],
            verbose=False
        )
```

```
        # predict and evaluate using Macro F1-Score
        preds = model.predict(X_val)
        fold_f1 = f1_score(y_val, preds, average='macro')
        f1_scores.append(fold_f1)

    return np.mean(f1_scores)

optuna.logging.set_verbosity(optuna.logging.WARNING)

study = optuna.create_study(direction='maximize')
study.optimize(obj_xgboost, n_trials=150)

print(f"Best Macro F1-Score: {study.best_trial.value:.4f}")
print(f"Best Parameters: {study.best_trial.params}")

# train final model on the best parameters found
best_params = study.best_trial.params
best_params['objective'] = 'multi:softmax'
best_params['num_class'] = 3
best_params['random_state'] = 42


final_model = xgb.XGBClassifier(**best_params)

# global_sample_weights = compute_sample_weight(class_weight=custom_weights,
global_sample_weights = compute_sample_weight(class_weight='balanced', y=y)

final_model.fit(X, y, sample_weight=global_sample_weights)
```

```
Class mapping: {'HARD': np.int64(0), 'MEDIUM': np.int64(1), 'SOFT': np.int64
(2)}
Best Macro F1-Score: 0.6524
Best Parameters: {'max_depth': 7, 'learning_rate': 0.010639743158486061, 'su
bsample': 0.9061120416105607, 'colsample_bytree': 0.5239087551434066, 'min_c
hild_weight': 2, 'gamma': 0.0004776400414104132, 'lambda': 0.310891261975017
15, 'alpha': 0.012287922862146427}
```

Out[7]:

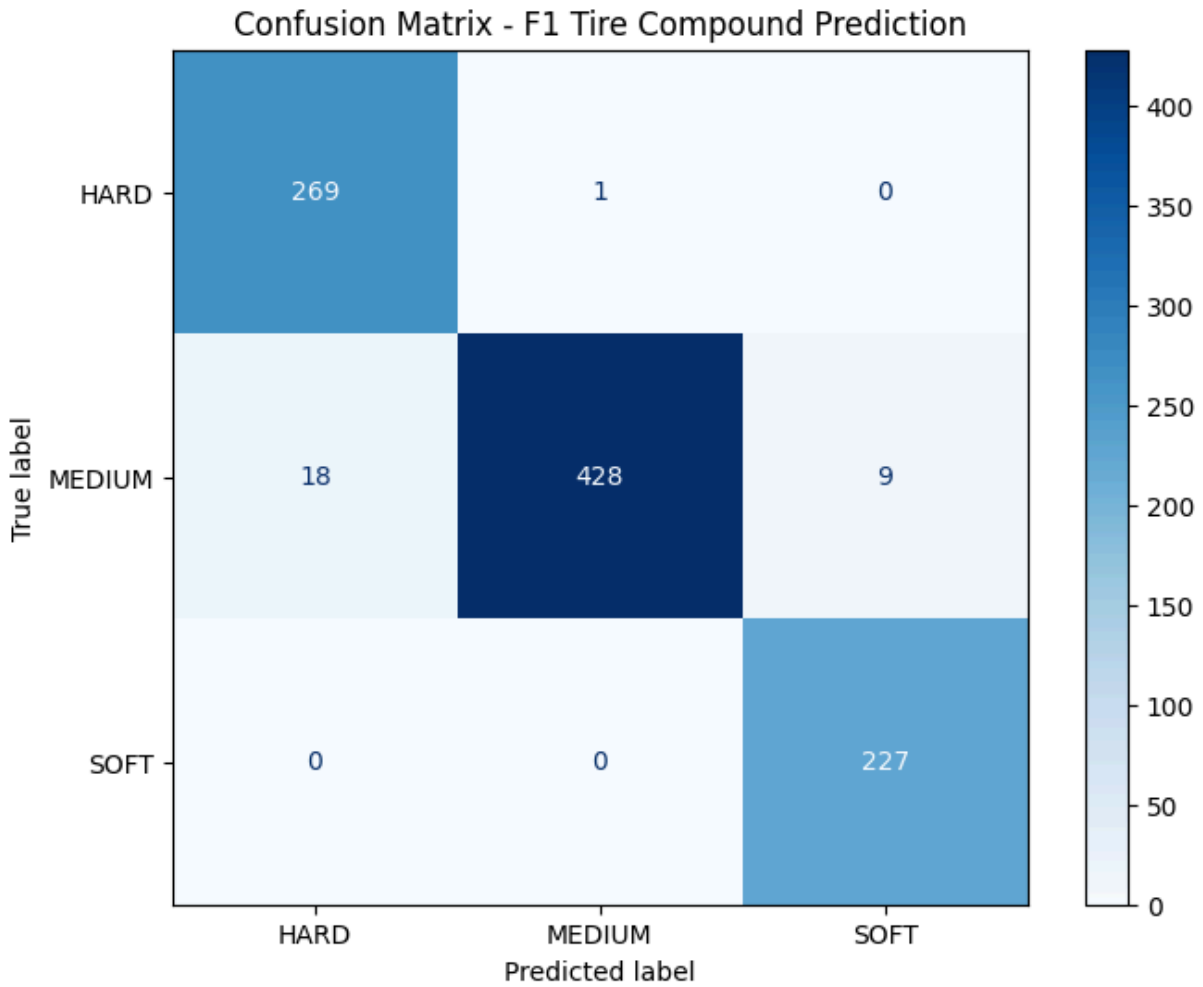
```
▾ XGBClassifier
▸ Parameters
```

In [8]:

```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, Confusi
import shap
import xgboost as xgb

##################################################################### EVALUA

y_pred = final_model.predict(X)

target_names = ['HARD', 'MEDIUM', 'SOFT']
print(classification_report(y, y_pred, target_names=target_names))

# CONFUSION MATRIX
```

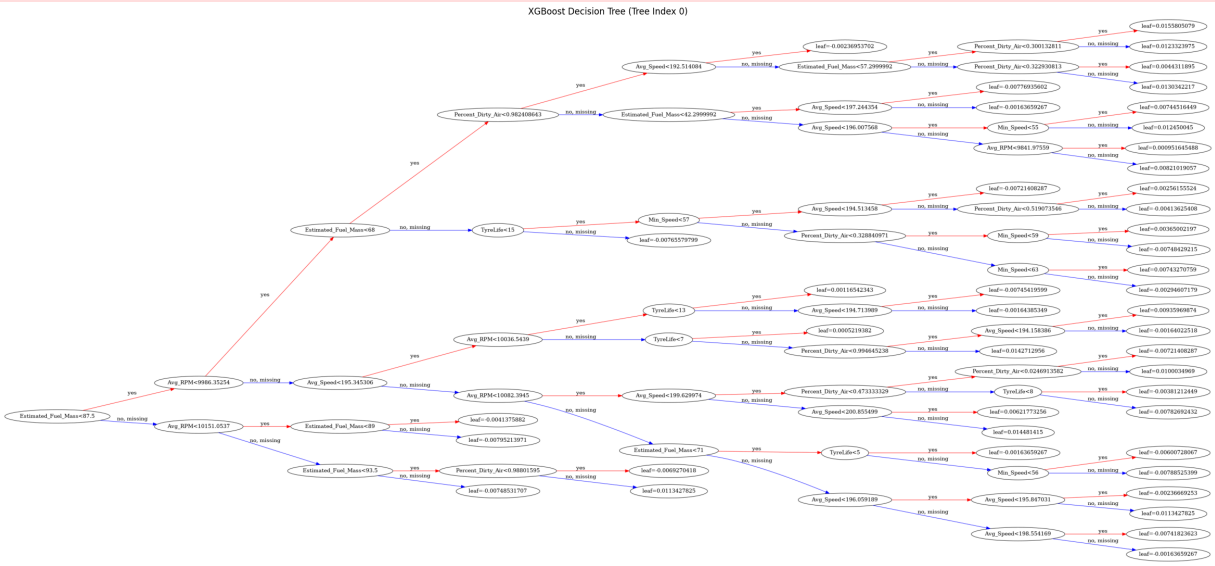
```
print("CONFUSION MATRIX")
cm = confusion_matrix(y, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_nam

fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(cmap='Blues', ax=ax)
plt.title("Confusion Matrix - F1 Tire Compound Prediction")
plt.show()


# XGBoost Tree Visualization
print("XGBOOST DECISION TREE")
final_model.get_booster().feature_names = list(X.columns)
fig, ax = plt.subplots(figsize=(30, 15))
xgb.plot_tree(final_model, num_trees=0, ax=ax, rankdir='LR')
plt.title("XGBoost Decision Tree (Tree Index 0)")
plt.show()


# SHAP Interpretability Plot
print("SHAP PLOT")
explainer = shap.TreeExplainer(final_model)
shap_values = explainer.shap_values(X)
shap.summary_plot(shap_values, X, class_names=target_names, plot_type="bar",
plt.show()
```

```
              precision    recall  f1-score   support

        HARD       0.94      1.00      0.97       270
      MEDIUM       1.00      0.94      0.97       455
        SOFT       0.96      1.00      0.98       227

    accuracy                           0.97       952
   macro avg       0.97      0.98      0.97       952
weighted avg       0.97      0.97      0.97       952

CONFUSION MATRIX
```

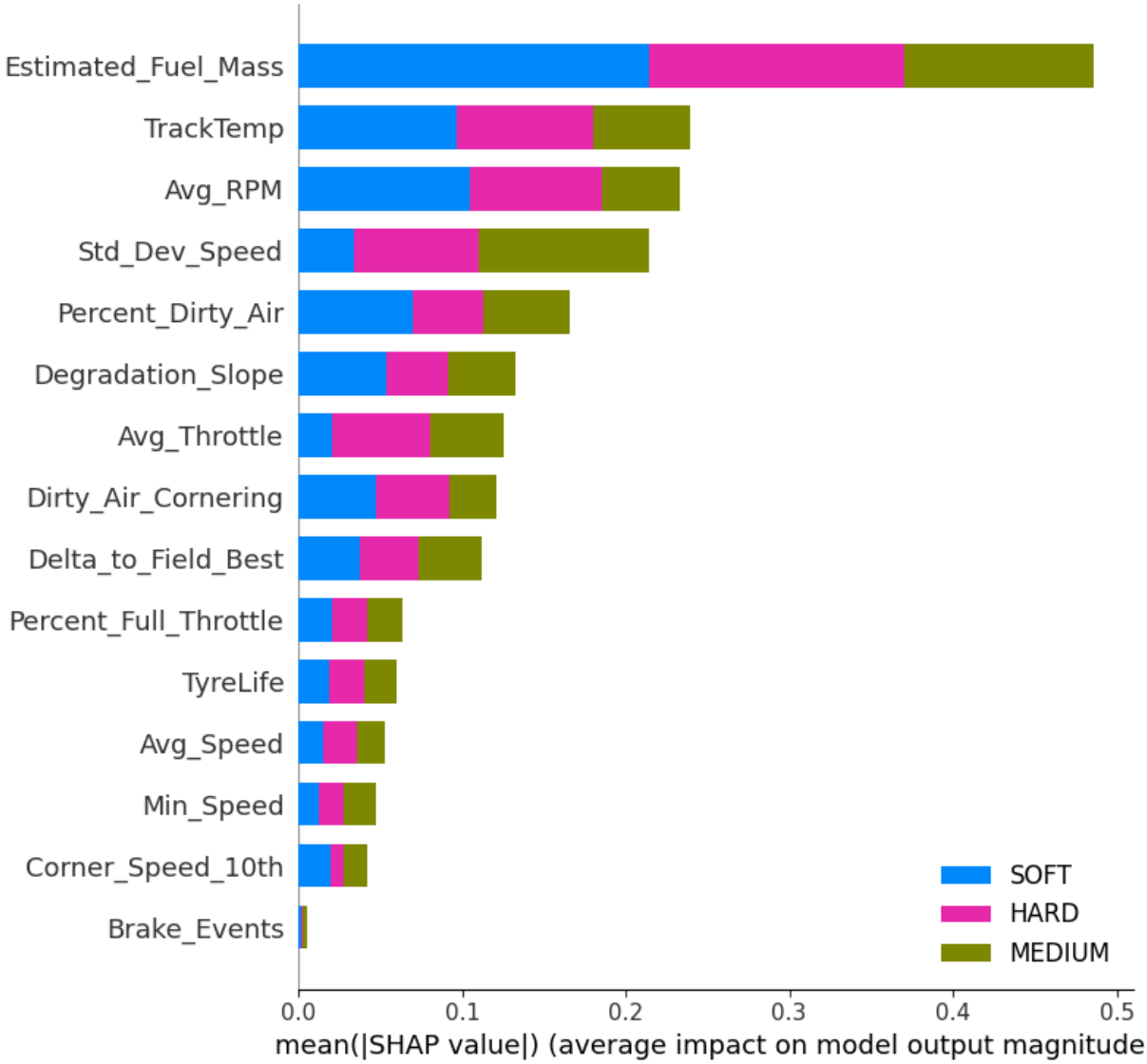


XGBOOST DECISION TREE

```
/home/c/Desktop/AI_MUAR/ai_f1/.venv/lib/python3.13/site-packages/xgboost/plo
tting.py:268: FutureWarning: The `num_trees` parameter is deprecated, use `t
ree_idx` insetad.
  warnings.warn(
```



SHAP PLOT

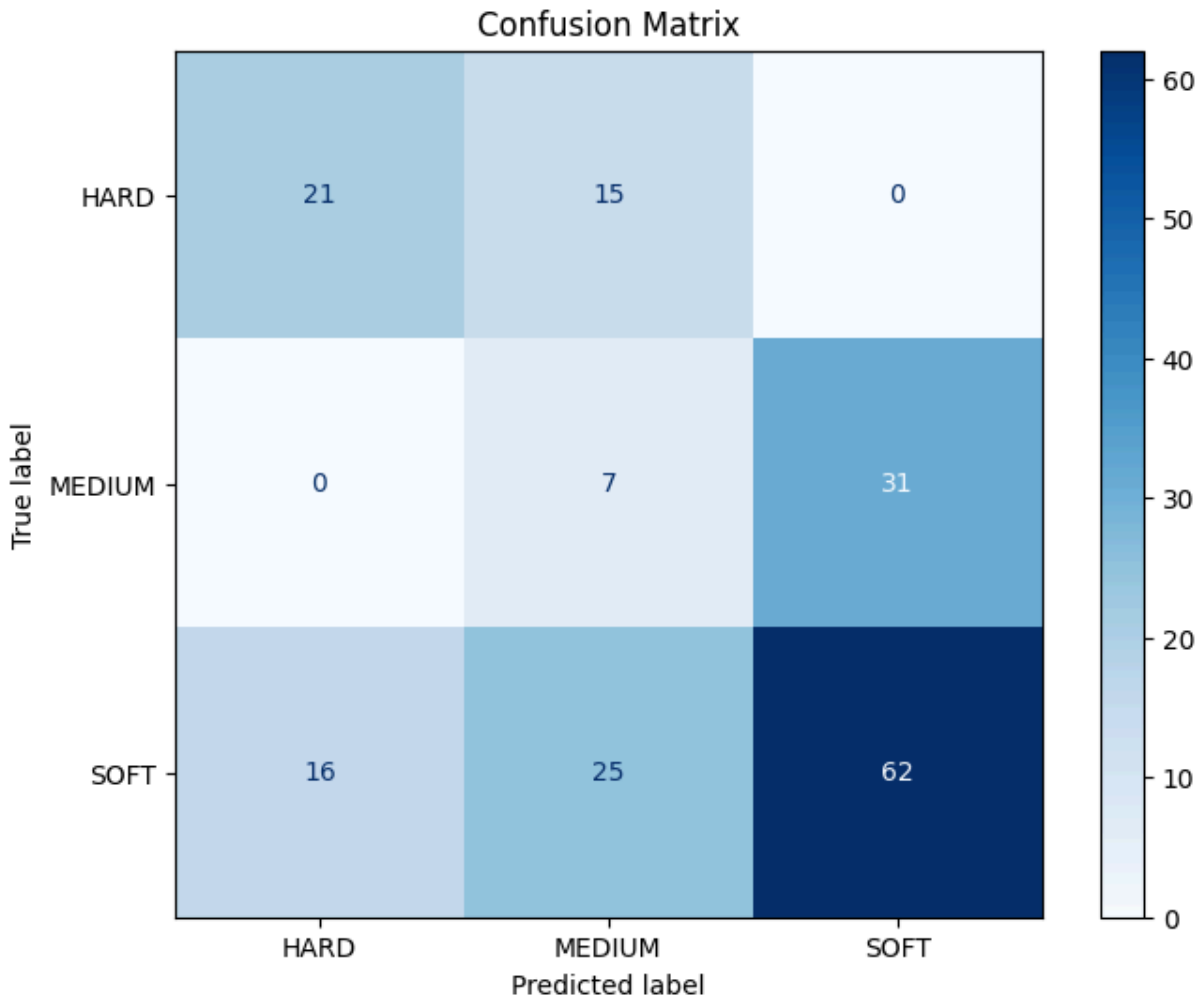


In [ ]:

In [9]:
```
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import classification_report, confusion_matrix, Confusi
from sklearn.utils.class_weight import compute_sample_weight
import xgboost as xgb
import shap
import matplotlib.pyplot as plt

################################################################### STRICT

# set up grouped split - 80% Train, 20% Test
gss = GroupShuffleSplit(test_size=0.20, n_splits=1, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups=groups))

X_train_strict = X.iloc[train_idx]
X_test_strict = X.iloc[test_idx]
y_train_strict = y.iloc[train_idx]
y_test_strict = y.iloc[test_idx]

print(f"TRAININING on {len(X_train_strict)} laps. TESTING on {len(X_test_str
```

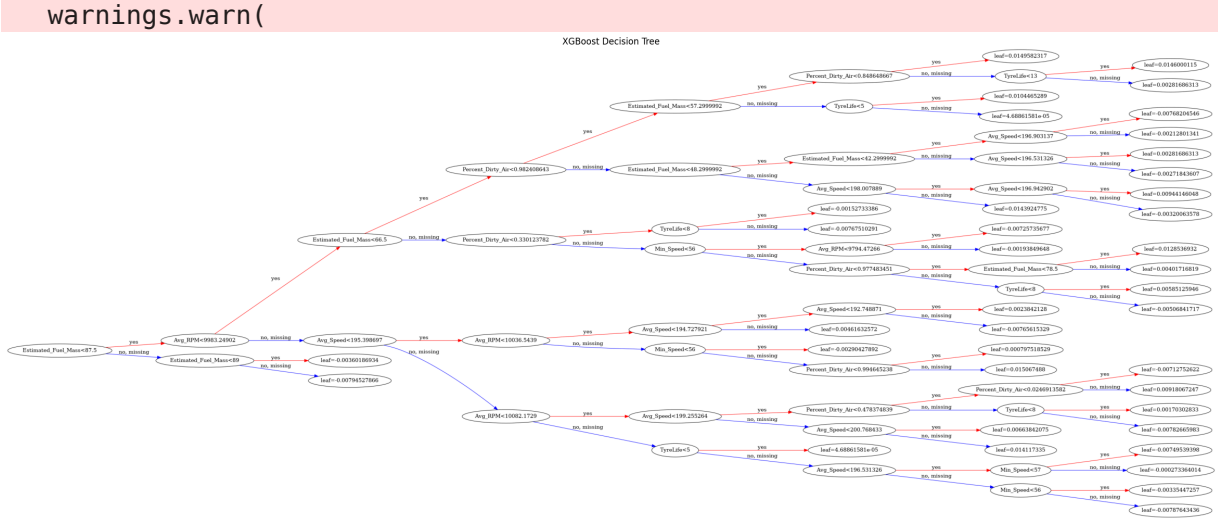
```
# calc sample weights ONLY on training data
# train_sample_weights = compute_sample_weight(class_weight=custom_weights,
train_sample_weights = compute_sample_weight(class_weight='balanced', y=y_tr

# initialize and train the model ONLY on the 80% training data
strict_model = xgb.XGBClassifier(**best_params)
strict_model.fit(X_train_strict, y_train_strict, sample_weight=train_sample_

# predict ONLY on the 20% holdout test data
y_pred_strict = strict_model.predict(X_test_strict)

print("\n TRUE UNSEEN CLASSIFICATION REPORT")
print(classification_report(y_test_strict, y_pred_strict, target_names=targe

print("\n CONFUSION MATRIX")
cm_strict = confusion_matrix(y_test_strict, y_pred_strict)
disp_strict = ConfusionMatrixDisplay(confusion_matrix=cm_strict, display_lab

fig, ax = plt.subplots(figsize=(8, 6))
disp_strict.plot(cmap='Blues', ax=ax)
plt.title("Confusion Matrix")
plt.show()

print("\n XGBOOST DECISION TREE")
strict_model.get_booster().feature_names = list(X.columns)
fig, ax = plt.subplots(figsize=(30, 15))
xgb.plot_tree(strict_model, num_trees=0, ax=ax, rankdir='LR')
plt.title("XGBoost Decision Tree")
plt.show()

print("\n SHAP INTERPRETABILITY")
explainer_strict = shap.TreeExplainer(strict_model)
shap_values_strict = explainer_strict.shap_values(X_test_strict)

shap.summary_plot(shap_values_strict, X_test_strict, class_names=target_name
```

```
TRAININING on 775 laps. TESTING on 177 unseen laps.

 TRUE UNSEEN CLASSIFICATION REPORT
              precision    recall  f1-score   support

        HARD       0.57      0.58      0.58        36
      MEDIUM       0.15      0.18      0.16        38
        SOFT       0.67      0.60      0.63       103

    accuracy                           0.51       177
   macro avg       0.46      0.46      0.46       177
weighted avg       0.54      0.51      0.52       177


 CONFUSION MATRIX
```

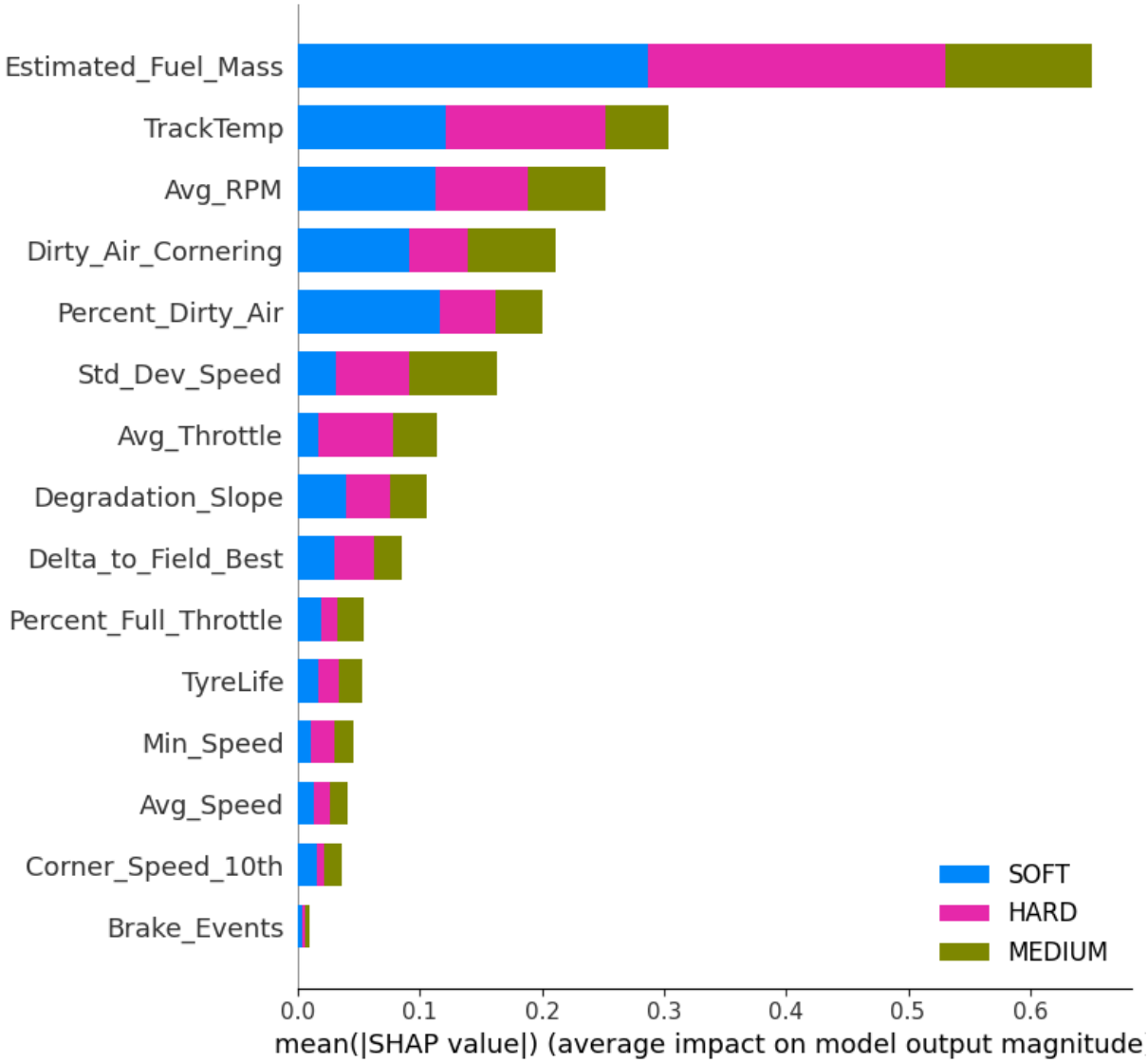


XGBOOST DECISION TREE

```
/home/c/Desktop/AI_MUAR/ai_f1/.venv/lib/python3.13/site-packages/xgboost/plo
tting.py:268: FutureWarning: The `num_trees` parameter is deprecated, use `t
ree_idx` insetad.
  warnings.warn(
```



SHAP INTERPRETABILITY


Estimated_Fuel_Mass
TrackTemp
Avg_RPM
Dirty_Air_Cornering
Percent_Dirty_Air
Std_Dev_Speed
Avg_Throttle
Degradation_Slope
Delta_to_Field_Best
Percent_Full_Throttle
TyreLife
Min_Speed
Avg_Speed
Corner_Speed_10th
Brake_Events
SOFT
HARD
MEDIUM
0.0
0.1
0.2
0.3
0.4
0.5
0.6
mean(|SHAP value|) (average impact on model output magnitude

In [ ]: